2024-11-05-NetFlows

Generated by Scribe.AI

December 31, 2024

Slide 1

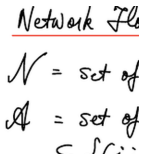
Content

Network Flow (IV) Chapter 14)
N = set of nodes $\{i\}$
$\mathcal{A}$ = set of (directed) arcs
$= \{(i,j) : i,j \in N\}$
Network = (N, A)
graph (or digraph)

Description

This slide introduces the concept of network flow in the context of a linear programming course. It defines a network as a tuple $(N, \mathcal{A})$, where N is the set of nodes and $\mathcal{A}$ is the set of directed arcs. The set N is explicitly defined as $\{i\}$, indicating that the nodes are indexed by i. The set $\mathcal{A}$ is defined as the set of ordered pairs (i,j) such that both i and j are in N, representing directed edges from node i to node j. The slide also includes a small illustrative graph depicting a network with several nodes and directed edges. The underlined portion emphasizes the definition of the set of nodes. The term "digraph" is used as a synonym for a directed graph. The context of linear programming is crucial because network flow problems are a significant application area within linear programming, often solved using techniques like the simplex method or other optimization algorithms.

Figures



(a) A small directed graph with several nodes and edges.

Slide 2

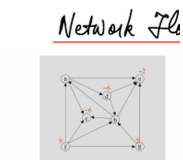
Content

Network Flow (IV) Chapter 14)
b_i
[scale=0.8] [circle,draw] (a) at (0,2) a; [circle,draw] (b) at (2,2) b; [circle,draw] (c) at (4,2) c; [circle,draw] (d) at (3,0) d; [circle,draw] (e) at (1,0) e; [circle,draw] (f) at (0,-2) f; [circle,draw] (g) at (4,-2) g;
[->] (a) – node[above] 8 (b); [->] (b) – node[above] -2 (c); [->] (b) – node[right] -6 (d); [->] (d) – node[below] 5 (e); [->] (e) – node[below] 9 (f); [->] (f) – node[left] 4 (a); [->] (f) – node[below] 7 (e); [->] (e) – node[left] -4 (a); [->] (a) – node[left] 3 (f); [->] (c) – node[right] 2 (b); [->] (c) – node[right] 2 (g); [->] (d) – node[right] 1 (g); [->] (g) – node[below] 5 (d);
c_{ij}
[scale=0.8] [circle,draw] (a) at (0,2) a; [circle,draw] (b) at (2,0) b; [circle,draw] (c) at (4,2) c; [circle,draw] (d) at (3,1) d; [circle,draw] (e) at (1,1) e; [circle,draw] (f) at (0,-2) f; [circle,draw] (g) at (4,-2) g;
[->] (a) – node[left] 56 (e); [->] (e) – node[left] 108 (f); [->] (f) – node[below] 48 (g); [->] (g) – node[right] 33 (d); [->] (d) – node[right] 19 (c); [->] (c) – node[above] 15 (d); [->] (d) – node[above] 38 (b); [->] (b) – node[above] 7 (d); [->] (b) – node[above] 28 (a); [->] (a) – node[above] 10 (c); [->] (e) – node[below] 48 (b); [->] (f) – node[below] 24 (g);
(1) $b_i > 0$ (supply) ; $b_i < 0$ (demand) $\sum_{i \in N} b_i = 0$
(2) x_{ij} = amount transported from i to j
(3) c_{ij} = cost of transportation from i to j
(4) $\min_{(i,j) \in \mathcal{A}} \sum c_{ij} x_{ij}$

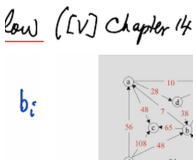
Description

This slide introduces network flow problems in the context of a linear programming course. It presents two figures: Figure 14.1 shows a directed graph representing a network with 7 nodes and 14 arcs. Numbers next to each node represent the supply ($b_i > 0$) or demand ($b_i < 0$) at that node, with the constraint that the total supply equals the total demand ($\sum_{i \in N} b_i = 0$). Figure 14.2 shows the same network, but with each arc labeled with a cost (c_{ij}). The slide then defines key variables: x_{ij} represents the amount of flow transported from node i to node j, and c_{ij} represents the cost per unit of flow on arc (i, j). Finally, the objective function is stated: minimize the total cost of transportation, $\min_{(i,j) \in \mathcal{A}} \sum c_{ij} x_{ij}$. The context of linear programming is crucial because network flow problems are a classic application of linear programming, often solved using techniques like the simplex method or other optimization algorithms. The slide sets up the problem formulation for a minimum cost network flow problem.

Figures



(a) A network flow graph with 7 nodes and 14 arcs. Numbers on nodes represent supply/demand.



(b) A network flow graph with 7 nodes and arcs labeled with costs.

Slide 3

Content

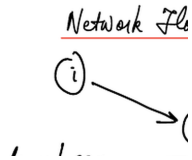
Network Flow (IV) Chapter 14)
[scale=0.8] [circle,draw] (i) at (0,2) i; [circle,draw] (k) at (2,1) k; [circle,draw] (j) at (4,2) j; [->] (i) – (k); [->] (k) – node[above] b_k (j);
Balanced eqn at node k: $\sum_i x_{ik} + b_k = \sum_j x_{kj}$

$\sum_i x_{ik} - \sum_j x_{kj} = -b_k$
min $\quad c^T x$
$Ax = -b, \quad x \geq 0$

Description

This slide, part of a Linear Programming course (Chapter 14), focuses on network flow problems. It presents a small network with three nodes (i, k, j) and two directed arcs. Node k has a supply/demand value of b_k. The core concept is the "balanced equation" at node k, which states that the total flow into node k ($\sum_i x_{ik} + b_k$) must equal the total flow out of node k ($\sum_j x_{kj}$). This is rewritten as a constraint: $\sum_i x_{ik} - \sum_j x_{kj} = -b_k$. The variables x_{ik} represent the flow from node i to node k. The slide then presents the overall optimization problem: minimize $c^T x$ subject to $Ax = -b$ and $x \geq 0$, where c is the cost vector, A is the constraint matrix derived from the network structure, x is the flow vector, and b is the vector of supply/demand values at each node. The red boxes highlight the key equations. The context of linear programming is crucial because this slide formulates a network flow problem as a linear program, setting the stage for solving it using linear programming techniques.

Figures



(a) A small directed graph with three nodes and two edges. The nodes are labeled i, k, and j. The edge from k to j is labeled with b_k.

Slide 4

Content

Network Flow (IV) Chapter 14)
[scale=0.8] [circle,draw] (a) at (0,2) a; [circle,draw] (b) at (2,2) b; [circle,draw] (c) at (4,2) c; [circle,draw] (d) at (3,0) d; [circle,draw] (e) at (1,0) e; [circle,draw] (f) at (0,-2) f; [circle,draw] (g) at (4,-2) g;

[->] (a) – node[above] 8 (b); [->] (b) – node[above] -2 (c); [->] (b) – node[right] -6 (d); [->] (d) – node[below] 5 (e); [->] (e) – node[below] 9 (f); [->] (f) – node[left] 4 (a); [->] (f) – node[below] 7 (e); [->] (e) – node[left] -4 (a); [->] (a) – node[left] 3 (f); [->] (c) – node[right] 2 (b); [->] (c) – node[right] 2 (g); [->] (d) – node[right] 1 (g); [->] (g) – node[below] 5 (d); b_i

$[scale = 0.8][circle, draw](a)at(0, 2)a; [circle, draw](b)at(2, 0)b; [circle, draw](c)at(4, 2)c; [circle, draw](d)at(3, 1)d; [circle, dra$

[->] (a) – node[left] 56 (e); [->] (e) – node[left] 108 (f); [->] (f) – node[below] 48 (g); [->] (g) – node[right] 33 (d); [->] (d) – node[right] 19 (c); [->] (c) – node[above] 15 (d); [->] (d) – node[above] 38 (b); [->] (b) – node[above] 7 (d); [->] (b) – node[above] 28 (a); [->] (a) – node[above] 10 (c); [->] (e) – node[below] 48 (b); [->] (f) – node[below] 24 (g); c_{ij}

$$\begin{pmatrix} x_{ac} & x_{ad} & x_{ae} & x_{ah} & x_{bc} & x_{bd} & x_{be} & x_{df} & x_{dg} & x_{ef} \\ x_{eg} & x_{fh} & x_{fg} & x_{ge} \end{pmatrix}^T$$

$$A = \begin{pmatrix} -1 & -1 & -1 & 1 & & & & & & \\ & & & & -1 & -1 & -1 & 1 & & \\ & & & & & & & & -1 & -1 \\ -1 & 1 & & & & & & & & \\ 1 & & & & 1 & & & & & \\ & 1 & & & & 1 & & & & \\ & & 1 & & & & 1 & & & \\ & & & & & & & 1 & 1 & \\ & & -1 & & & & & & & \\ & & & & & & & & & 1 \\ 1 & & 1 & & & & & & & \end{pmatrix} \quad b = \begin{pmatrix} 0 \\ 0 \\ -6 \\ -6 \\ -2 \\ 9 \\ 5 \end{pmatrix}$$

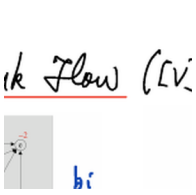
$$c^T = \begin{pmatrix} 48 & 28 & 10 & 7 & 65 & 7 & 38 & 15 & 56 & 48 \\ 108 & 24 & 33 & 19 \end{pmatrix}$$

Description

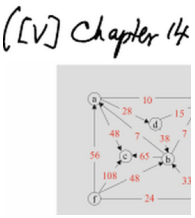
This slide, from a Linear Programming course (Chapter 14), presents a minimum-cost network flow problem. Two figures depict a network with 7 nodes and 14 arcs: one shows supply/demand at each node (Figure 14.1), and the other shows the cost of each arc (Figure 14.2). The supply/demand values are represented by the vector b, where positive values indicate supply and negative values indicate demand. The costs are represented by the vector c. The incidence matrix A is a 7x14 matrix representing the network's structure. Each column corresponds to an arc, and each row corresponds to a node. A '1' in entry (i,j) indicates that arc j starts at node i, a '-1' indicates that arc j ends at node i, and a '0' indicates that arc j is not incident to node i. The objective is to minimize the total cost of flow, which can be expressed as $c^T x$, subject to the constraints $Ax = b$ and $x \geq 0$, where x is the vector of flows on each arc. The slide explicitly defines the incidence matrix A, the cost vector c, and the supply/demand vector b. The context of linear programming is crucial because this slide formulates a network flow problem as a linear program, ready to be solved using linear programming techniques like the simplex method.

Figures

(a) A network flow graph with 7 nodes and 14 arcs. Numbers on nodes represent supply/demand.



(b) A network flow graph with 7 nodes and arcs labeled with costs.



(c) Incidence matrix and supply/demand vector.

Slide 5

Content

Network Flow (IV) Chapter 14)
[scale=0.8] [circle,draw] (a) at (0,2) 8; [circle,draw] (b) at (2,2) 0; [circle,draw] (c) at (4,2) -2; [circle,draw] (d) at (3,0) -6; [circle,draw] (e) at (1,0) 0; [circle,draw] (f) at (0,-2) 4; [circle,draw] (g) at (4,-2) 5;

[->] (a) – node[above] 8 (b); [->] (b) – node[above] -2 (c); [->] (b) – node[right] -6 (d); [->] (d) – node[below] 5 (e); [->] (e) – node[below] 9 (f); [->] (f) – node[left] 4 (a); [->] (f) – node[below] 7 (e); [->] (e) – node[left] -4 (a); [->] (a) – node[left] 3 (f); [->] (c) – node[right] 2 (b); [->] (c) – node[right] 2 (g); [->] (d) – node[right] 1 (g); [->] (g) – node[below] 5 (d); [scale=0.8] [circle,draw] (a) at (0,2) ; [circle,draw] (b) at (2,0) ; [circle,draw] (c) at (4,2) ; [circle,draw] (d) at (3,1) ; [circle,draw] (e) at (1,1) ; [circle,draw] (f) at (0,-2) ; [circle,draw] (g) at (4,-2) ;

[->] (a) – node[left] 56 (e); [->] (e) – node[left] 108 (f); [->] (f) – node[below] 48 (g); [->] (g) – node[right] 33 (d); [->] (d) – node[right] 19 (c); [->] (c) – node[above] 15 (d); [->] (d) – node[above] 38 (b); [->] (b) – node[above] 7 (d); [->] (b) – node[above] 28 (a); [->] (a) – node[above] 10 (c); [->] (e) – node[below] 48 (b); [->] (f) – node[below] 24 (g);

$$A_{i,(k,l)} = \begin{cases} -1 & i = k \\ 1 & i = l \\ 0 & i \neq k, l \end{cases}$$

Description

This slide (Slide 5 of 22) in a Linear Programming course (Chapter 14) continues the discussion of network flow problems. It shows two figures: Figure 14.1 depicts a network with 7 nodes and 14 arcs, with numbers next to each node indicating supply (positive) or demand (negative). Figure 14.2 shows the same network but with costs associated with each arc. The main content of the slide introduces the incidence matrix A, which is defined mathematically as: $A_{i,(k,l)} = \begin{cases} -1 & i = k \\ 1 & i = l \\ 0 & i \neq k, l \end{cases}$. This matrix represents the network's structure, where each column corresponds to an arc (k, l), and each row corresponds to a node i. A -1 indicates the arc's tail node, a 1 indicates the arc's head node, and 0 indicates the node is not incident to the arc. This sets the stage for formulating the network flow problem as a linear program, using the incidence matrix to represent the flow conservation constraints at each node. The context of linear programming is crucial because this slide provides the fundamental building block (the incidence matrix) for constructing the constraint matrix of the linear program that solves the minimum-cost network flow problem.

Figures

(a) A network flow graph with 7 nodes and 14 arcs. Numbers on nodes represent supply/demand.

(b) A network flow graph with 7 nodes and arcs labeled with costs.

Slide 6

Content

Network Flow (IV) Chapter 14)
[scale=0.8] [circle,draw] (a) at (0,2) ; [circle,draw] (b) at (2,0) ; [circle,draw] (c) at (4,2) ; [circle,draw] (d) at (3,1) ; [circle,draw] (e) at (1,1) ; [circle,draw] (f) at (0,-2) ; [circle,draw] (g) at (4,-2) ;
[->] (a) – node[left] 56 (e); [->] (e) – node[left] 108 (f); [->] (f) – node[below] 48 (g); [->] (g) – node[right] 33 (d); [->] (d) – node[right] 19 (c); [->] (c) – node[above] 15 (d); [->] (d) – node[above] 38 (b); [->] (b) – node[above] 7 (d); [->] (b) – node[above] 28 (a); [->] (a) – node[above] 10 (c); [->] (e) – node[below] 48 (b); [->] (f) – node[below] 24 (g); b_i
$[scale = 0.8][circle, draw](a)at(0, 2); [circle, draw](b)at(2, 0); [circle, draw](c)at(4, 2); [circle, draw](d)at(3, 1); [circle, draw](e$
[->] (a) – node[left] 56 (e); [->] (e) – node[left] 108 (f); [->] (f) – node[below] 48 (g); [->] (g) – node[right] 33 (d); [->] (d) – node[right] 19 (c); [->] (c) – node[above] 15 (d); [->] (d) – node[above] 38 (b); [->] (b) – node[above] 7 (d); [->] (b) – node[above] 28 (a); [->] (a) – node[above] 10 (c); [->] (e) – node[below] 48 (b); [->] (f) – node[below] 24 (g); c_{ij}
$\text{Rank}(A) = m - 1$
Sum of all rows $= 0$ $(\text{Rank}(A) \leq m - 1)$
Can delete one row (Root node)

Description

This slide (Slide 6 of 22), within the context of a Linear Programming course (Chapter 14 on Network Flow), discusses the rank of the incidence matrix A associated with a network flow problem. Two figures depict the same network graph: one shows the supply/demand at each node (b_i), and the other shows the cost of each arc (c_{ij}). The key takeaway is that the rank of the incidence matrix A is $m - 1$, where m is the number of nodes. This is because the sum of all rows of A is zero (flow conservation), implying linear dependence among the rows. Consequently, one row can be deleted without losing information (often, a row corresponding to a root node is removed). This property is crucial for solving network flow problems using linear programming techniques, as it affects the structure and solvability of the resulting linear system. The notation $\text{Rank}(A)$ denotes the rank of matrix A, and m represents the number of nodes in the network.

Figures

(a) A network flow graph with 7 nodes and arcs labeled with costs b_i.

(b) A network flow graph with 7 nodes and arcs labeled with costs c_{ij}.

Slide 7

Content

Network Flow (IV) Chapter 14)

$$\text{(P)} \quad \begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax = -b, \quad x \geq 0 \end{aligned}$$

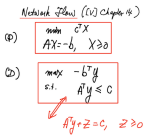
$$\text{(D)} \quad \begin{aligned} \max \quad & -b^T y \\ \text{s.t.} \quad & A^T y \leq c \end{aligned}$$

$$A^T y + z = c, \quad z \geq 0$$

Description

This slide (Slide 7 of 22) from a Linear Programming course (Chapter 14 on Network Flows) presents the primal (P) and dual (D) linear programming formulations of a minimum-cost network flow problem. The primal problem (P) minimizes the total cost ($c^T x$) subject to flow conservation constraints ($Ax = -b$) and non-negativity constraints ($x \geq 0$). Here, c is the vector of arc costs, x is the vector of arc flows, A is the incidence matrix of the network, and b is the vector of node supplies/demands. The dual problem (D) maximizes $-b^T y$, where y is the vector of node potentials, subject to the constraint $A^T y \leq c$. The additional constraint $A^T y + z = c, z \geq 0$ is included, introducing slack variables z to convert the inequality constraint into an equality constraint. The arrow indicates the relationship between the primal and dual problems, highlighting the complementary slackness conditions implicitly. The context of linear programming is essential because this slide shows how a network flow problem can be formulated as a linear program and its dual, allowing for the application of duality theory and optimization algorithms to solve it.

Figures



(a)

Slide 8

Content

Network Flow (IV) Chapter 14)

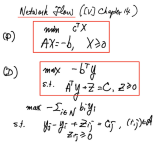
$$
\begin{aligned}
&\text{(P)} && \min \quad c^T x \\
& && \text{s.t.} \quad Ax = -b, \quad x \geq 0 \\
&\text{(D)} && \max \quad -b^T y \\
& && \text{s.t.} \quad A^T y + z = c, \quad z \geq 0
\end{aligned}
$$

$$
\begin{aligned}
&\max \quad -\sum_{i \in N} b_i y_i \\
&\text{s.t.} \quad y_j - y_i + z_{ij} = c_{ij}, \quad (i,j) \in A \\
&z_{ij} \geq 0
\end{aligned}
$$

Description

Slide 8 of 22 in a Linear Programming course (Chapter 14, Network Flows) presents the primal (P) and dual (D) linear programs for the minimum-cost network flow problem. The primal problem minimizes the total cost ($c^T x$) subject to flow conservation ($Ax = -b$) and non-negativity ($x \geq 0$). The dual problem is presented in two equivalent forms. The first is compact, maximizing $-b^T y$ subject to $A^T y + z = c$ and $z \geq 0$. The second form expands the dual problem, explicitly showing the objective function as the maximization of $-\sum_{i \in N} b_i y_i$, where N is the set of nodes. The constraints are rewritten as $y_j - y_i + z_{ij} = c_{ij}$ for each arc (i, j) in the arc set A, with non-negativity constraints on the dual slack variables z_{ij}. The notation is consistent with previous slides, where c represents arc costs, x represents arc flows, A is the incidence matrix, b represents node supplies/demands, and y represents node potentials. This slide emphasizes the alternative representations of the dual problem, providing flexibility in solving and interpreting the network flow problem within the context of linear programming.

Figures



(a)

Slide 9

Content

Network Flow (IV) Chapter 14)

$$\begin{aligned} &\min \quad c^T x \\ (\mathrm{P}) \quad &\\ &\text{s.t.} \quad Ax = -b, \quad x \geq 0 \end{aligned}$$

$$\begin{aligned} &\max \quad -b^T y \\ (\mathrm{D}) \quad &\\ &\text{s.t.} \quad A^T y + z = c, \quad z \geq 0 \end{aligned}$$

$$\max \quad -\sum_{i \in N} b_i y_i$$

$$\text{s.t.} \quad y_j - y_i + z_{ij} = c_{ij}, \quad (i,j) \in A$$

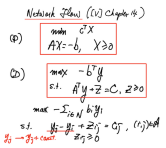
$y_j \to y_j + \text{const.}$

$z_{ij} \geq 0$

Description

Slide 9 of 22 in a Linear Programming course (Chapter 14, Network Flows) shows the primal (P) and dual (D) linear programming formulations of a minimum-cost network flow problem. The primal minimizes the total cost ($c^T x$) subject to flow conservation ($Ax = -b$) and non-negativity ($x \geq 0$). The dual maximizes $-b^T y$ subject to $A^T y + z = c$ and $z \geq 0$. The dual is also expressed in a more explicit form, maximizing $-\sum_{i \in N} b_i y_i$ subject to $y_j - y_i + z_{ij} = c_{ij}$ for each arc (i,j) in the arc set A, and $z_{ij} \geq 0$. Crucially, a note is added indicating that the dual variables y_j are only determined up to an additive constant; adding a constant to all y_j does not change the feasibility or optimality of the solution. This is because only the *differences* $y_j - y_i$ are relevant in the constraints. The notation is consistent with previous slides: c are arc costs, x are arc flows, A is the incidence matrix, b are node supplies/demands, y are node potentials, and z are dual slack variables. The slide highlights the flexibility in representing the dual problem and the inherent ambiguity in the absolute values of the dual variables y_j in the context of network flow problems.

Figures



(a)

Slide 10

Content

Spanning Trees and Bases

[scale=0.7] [circle,draw,fill=blue!50] (1) at (0,0) ; [circle,draw,fill=blue!50] (2) at (1,1) ; [circle,draw,fill=blue!50] (3) at (2,0) ; [circle,draw,fill=blue!50] (4) at (1,-1) ; [circle,draw,fill=blue!50] (5) at (3,1) ; (1)–(2)–(3)–(4)–(1); (2)–(4); [scale=0.7] [circle,draw,fill=blue!50] (1) at (0,0) ; [circle,draw,fill=blue!50] (2) at (1,1) ; [circle,draw,fill=blue!50] (3) at (2,0) ; [circle,draw,fill=blue!50] (4) at (1,-1) ; [circle,draw,fill=blue!50] (5) at (3,1) ; (1)–(2)–(3)–(4)–(1); [scale=0.7] [circle,draw,fill=blue!50] (1) at (0,0) ; [circle,draw,fill=blue!50] (2) at (1,1) ; [circle,draw,fill=blue!50] (3) at (2,0) ; [circle,draw,fill=blue!50] (4) at (1,-1) ; [circle,draw,fill=blue!50] (5) at (3,1) ; (1)–(2)–(3)–(4)–(1);

[scale=0.7] [circle,draw,fill=blue!50] (1) at (0,0) ; [circle,draw,fill=blue!50] (2) at (1,1) ; [circle,draw,fill=blue!50] (3) at (2,0) ; [circle,draw,fill=blue!50] (4) at (1,-1) ; [circle,draw,fill=blue!50] (5) at (3,1) ; (1)–(2)–(3)–(4)–(1); (2)–(4); (3)–(5); [scale=0.7] [circle,draw,fill=blue!50] (1) at (0,0) ; [circle,draw,fill=blue!50] (2) at (1,1) ; [circle,draw,fill=blue!50] (3) at (2,0) ; [circle,draw,fill=blue!50] (4) at (1,-1) ; [circle,draw,fill=blue!50] (5) at (3,1) ; (1)–(2)–(3)–(4)–(1);

- Path = $(n_1, n_2, \ldots, n_k)$,
- for all l, (n_l, n_{l+1}) or $(n_{l+1}, n_l) \in \mathcal{A}$
- Cycle: $(n_1, n_2, \ldots, n_k)$, $n_k = n_1$
- Tree: connected, acyclic
- Spanning tree $T = (N, \hat{A})$, tree and $\hat{N} = N$

Description

Slide 10 of 22, within a Linear Programming course (Chapter 14, likely on Network Flows), introduces spanning trees and bases. The slide begins with two figures (Figures 14.3 and 14.4) illustrating connected and unconnected networks, and networks with and without cycles. Figure 14.3 shows examples of connected and unconnected networks. Figure 14.4 shows examples of networks with and without cycles. The main content defines key graph theory concepts:

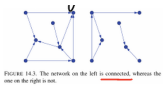
- **Path:** A sequence of nodes $(n_1, n_2, \ldots, n_k)$ where consecutive nodes are connected by an edge.
- **Cycle:** A path where the starting and ending nodes are the same ($n_k = n_1$).
- **Tree:** A connected graph with no cycles (acyclic).
- **Spanning Tree:** A subgraph $T = (N, \hat{A})$ that is a tree and includes all nodes (N) of the original graph. $\hat{A}$ represents the edges of the spanning tree.

These definitions are fundamental for understanding network flow algorithms and their application in linear programming problems. The notation $\mathcal{A}$ likely represents the set of arcs or edges in the network.

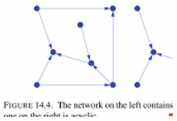
Figures

(a)

(b)



(c)



(d)

Slide 11

Content

Spanning Trees and Bases
- Balanced Flow: $AX = b$
- Feasible Flow: $X \geq 0$
- Tree Solution: $X_{(ij)} = 0$ if $(ij) \in Tree$

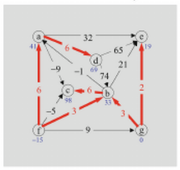
Description

Slide 11 of 22 in a Linear Programming course (Chapter 14, Network Flows) introduces the concepts of spanning trees and bases in the context of network flow problems. The slide defines three key terms:

- **Balanced Flow:** This refers to the flow conservation constraints in a network flow problem, represented by the matrix equation $AX = b$, where A is the incidence matrix, X is the vector of arc flows, and b is the vector of node supplies/demands.
- **Feasible Flow:** A flow is feasible if it satisfies the flow conservation constraints ($AX = b$) and the non-negativity constraints ($X \geq 0$).
- **Tree Solution:** This refers to a basic feasible solution where the flow on arcs not in a spanning tree is zero ($X_{(ij)} = 0$ if $(ij) \notin Tree$). A spanning tree is a subgraph that connects all nodes without forming any cycles.

The figure (Figure 14.6) illustrates a network with a spanning tree highlighted. The numbers on the arcs of the spanning tree represent the primal flows (X), the numbers next to the nodes represent the dual variables (y), and the numbers on the arcs not in the spanning tree represent the dual slacks (z). The slide connects the algebraic representation of network flows with the graphical representation using spanning trees, setting the stage for algorithms that leverage this connection to solve network flow problems efficiently.

Figures



(a)

Slide 12

Content

Spanning Trees and Bases
• Balanced Flow: $AX = b$
• Feasible Flow: $X \geq 0$
• Tree Solution: $X_{(ij)} = 0$ if $(ij) \notin Tree$
might not be feasible

Description

Slide 12 of 22 in a Linear Programming course (Chapter 14, Network Flows) discusses spanning trees and bases in the context of network flow problems. It builds upon the previous slide by introducing the concept of a "tree solution." The slide lists three key points:

- **Balanced Flow:** The flow conservation constraints are represented by $AX = b$, where A is the incidence matrix, X is the flow vector, and b is the supply/demand vector.
- **Feasible Flow:** A flow is feasible if it satisfies both flow conservation ($AX = b$) and non-negativity ($X \geq 0$).
- **Tree Solution:** A tree solution is a flow where the flow on arcs not in a spanning tree is zero ($X_{(ij)} = 0$ if $(ij) \notin Tree$). This is a basic solution.

Importantly, the slide notes that a tree solution might not be feasible, meaning it might violate the non-negativity constraint ($X \geq 0$). The figure (Figure 14.6) shows a network with a spanning tree highlighted. The numbers on the arcs within the spanning tree represent primal flows (X), the numbers next to the nodes represent dual variables (y), and the numbers on arcs outside the spanning tree represent dual slacks (z). The slide emphasizes that while a tree solution provides a basic solution, it doesn't guarantee feasibility.

Figures

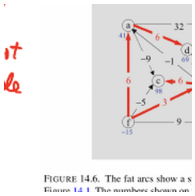


FIGURE 14.6. The fat arcs show a s

(a)

Slide 13

Content

Spanning Trees and Bases [scale=0.8] [circle,draw] (a) at (0,2) a; [circle,draw] (b) at (3,0) b; [circle,draw] (c) at (0,-1) c; [circle,draw] (d) at (2,1) d; [circle,draw] (e) at (4,2) e; [circle,draw] (f) at (0,-3) f; [circle,draw] (g) at (4,-2) g;

[->, very thick, red] (a) – node[above] 32 (e); [->, very thick, red] (a) – node[left] 41 (f); [->, very thick, red] (f) – node[below] 3 (b); [->, very thick, red] (f) – node[below] 9 (g); [->, very thick, red] (g) – node[right] 2 (e); [->, very thick, red] (b) – node[right] 3 (g); [->, very thick, red] (a) – node[above] 6 (d); [->, very thick, red] (d) – node[right] 21 (e); [->, very thick, red] (d) – node[right] 74 (b); [->, very thick, red] (c) – node[above] 6 (b); [->, very thick, red] (c) – node[left] 6 (a); [->, very thick, red] (f) – node[left] 6 (a); [->] (c) – node[above] 1 (d); [->] (c) – node[left] -9 (a); [->] (f) – node[below] -5 (c); [->] (f) – node[below] 9 (g);

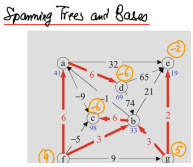
[circle,draw,orange] at (0,-3) 4; [circle,draw,orange] at (0,-1) 6; [circle,draw,orange] at (2,1) 6; [circle,draw,orange] at (4,2) 2; [circle,draw,orange] at (4,-2) 5;

[blue] at (0,-3.5) -15; [blue] at (0,1.5) 41; [blue] at (3, -0.5) 33; [blue] at (2, 0.5) 69; [blue] at (0, -1.5) 98; [blue] at (4, 1.5) 19; [blue] at (4, -2.5) 0;

Description

Slide 13 of 22, in a Linear Programming course (Chapter 14, Network Flows), presents a network flow problem represented graphically. The network consists of nodes (a, b, c, d, e, f, g) and directed arcs connecting them. Each arc has a capacity (number on the arc) and a flow (indicated by the thickness and color of the arc; thicker red arcs indicate positive flow). Some nodes have a supply or demand (numbers next to the nodes in blue). The orange circles next to some nodes represent dual variables (or shadow prices). The network is used to illustrate the concept of spanning trees and bases in the context of network flow optimization. The thick red arcs likely represent a basic feasible solution, where the flows on arcs not in a spanning tree are zero. The numbers on the arcs represent the flow values, and the numbers next to the nodes represent the net supply/demand at each node. The orange circles likely represent the dual variables associated with each node in the optimal solution. The slide visually demonstrates the relationship between a network flow problem, its graphical representation, and the underlying linear algebra (spanning trees and bases).

Figures



(a)

Slide 14

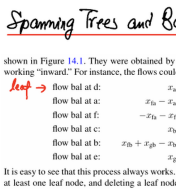
Content

Spanning Trees and Bases
shown in Figure 14.1. They were obtained by starting at the "leaves" of the tree and working "inward." For instance, the flows could be solved for successively as follows:
leaf $\rightarrow$ flow bal at d: $x_{ad} = 6$,
flow bal at a: $x_{ia} - x_{ad} = 0 \implies x_{ia} = 6$,
flow bal at f: $-x_{fa} + x_{fb} = -9 \implies x_{fb} = 3$,
flow bal at c: $x_{bc} = 6$,
flow bal at b: $x_{fb} + x_{gb} - x_{bc} = 0 \implies x_{gb} = 3$,
flow bal at e: $x_{ge} = 2$.
It is easy to see that this process always works. The reason is that every tree must have at least one leaf node, and deleting a leaf node together with the edge leading into it **Primal Flow**

Description

Slide 14 of 22 in a Linear Programming course (Chapter 14, Network Flows) explains how to find a primal flow solution using a spanning tree. The slide describes a method to solve for the flows in a network by starting at the "leaves" (nodes with only one edge connected) of a spanning tree and working inwards. The equations shown demonstrate how flow balance constraints at each node are used to iteratively solve for the flow on each arc. The notation x_{ij} represents the flow on the arc from node i to node j. The equations are derived from the flow conservation constraints ($AX = b$) where A is the incidence matrix, X is the vector of arc flows, and b is the vector of node supplies/demands. The process is illustrated with an example, showing how the flow on each arc is determined sequentially. The final paragraph explains why this process always works: every tree has at least one leaf node, and removing a leaf node and its incident edge simplifies the problem, allowing for iterative solution. The term "Primal Flow" is highlighted, indicating that this method solves for the primal variables (arc flows) in the network flow problem. The reference to Figure 14.1 suggests that a graphical representation of the network and its spanning tree is shown in a separate figure.

Figures



(a)

Slide 15

Content

Spanning Trees and Bases
The above computation suggests that spanning trees are related to bases in the simplex method. Let us pursue this idea. Normally, a basis is an invertible square submatrix of the constraint matrix. But for incidence matrices, no such submatrix exists. To see why, note that if we sum together all the rows of A, we get a row vector of all zeros (since each column of A has exactly one +1 and one -1). Of course, every square submatrix of A has this same property and so is singular. In fact, we shall show in a moment that for a connected network, there is exactly one redundant equation (i.e., the rank of A is exactly $m - 1$).
Let us select some node, say, the last one, and delete the flow balance constraint associated with this node from the constraints defining the problem (since it is redundant anyway). Let us call this node the root node. Let $\tilde{A}$ denote the incidence matrix A without the row corresponding to the root node (i.e., the last row), and let $\tilde{b}$ denote the supply/demand vector with the last entry deleted. The most important property of network flow problems is summarized in the following theorem:
THEOREM 14.1. A square submatrix of $\tilde{A}$ is a basis if and only if the arcs to which its columns correspond form a spanning tree.

Description

Slide 15 of 22 in a Linear Programming course (Chapter 14, Network Flows) discusses the relationship between spanning trees and bases in the context of network flow problems. It begins by explaining why a standard basis definition (invertible square submatrix) doesn't directly apply to incidence matrices of network flows. The key reason is that summing all rows of the incidence matrix A always results in a zero vector, implying singularity of any square submatrix. The slide then introduces a modified approach: selecting a root node, deleting the corresponding row from the incidence matrix A (resulting in $\tilde{A}$), and deleting the corresponding entry from the supply/demand vector b (resulting in $\tilde{b}$). Finally, Theorem 14.1 is presented, stating that a square submatrix of $\tilde{A}$ forms a basis if and only if the corresponding arcs in the network constitute a spanning tree. This theorem establishes a fundamental connection between the linear algebra of the problem (bases) and the graph theory of the network (spanning trees).

Figures

(a)

Slide 16

Content

Spanning Trees and Bases
The above computation suggests that spanning trees are related to bases in the simplex method. Let us pursue this idea. Normally, a basis is an invertible square submatrix of the constraint matrix. But for incidence matrices, no such submatrix exists. To see why, note that if we sum together all the rows of A, we get a row vector of all zeros (since each column of A has exactly one +1 and one -1). Of course, every square submatrix of A has this same property and so is singular. In fact, we shall show in a moment that for a connected network, there is exactly one redundant equation (i.e., the rank of A is exactly $m - 1$).
Let us select some node, say, the last one, and delete the flow balance constraint associated with this node from the constraints defining the problem (since it is redundant anyway). Let us call this node the root node. Let $\tilde{A}$ denote the incidence matrix A without the row corresponding to the root node (i.e., the last row), and let $\tilde{b}$ denote the supply/demand vector with the last entry deleted. The most important property of network flow problems is summarized in the following theorem:
THEOREM 14.1. A square submatrix of $\tilde{A}$ is a basis if and only if the arcs to which its columns correspond form a spanning tree.
(if : $B\tilde{x}B = -\tilde{b}$ only if : Ex. 14.12)

Description

Slide 16 of 22, in a Linear Programming course (Chapter 14, Network Flows), delves into the relationship between spanning trees and bases in the context of network flow problems. It explains that while a basis is typically defined as an invertible square submatrix of the constraint matrix, this definition doesn't directly apply to incidence matrices due to their inherent singularity (the sum of rows is always zero). The slide then proposes a solution: selecting a root node and removing its corresponding row from the incidence matrix A to create $\tilde{A}$, and similarly removing the corresponding entry from the supply/demand vector b to create $\tilde{b}$. Theorem 14.1 is then presented, stating that a square submatrix of $\tilde{A}$ is a basis if and only if the corresponding arcs form a spanning tree in the network. The equation $B\tilde{x}B = -\tilde{b}$ is mentioned, likely representing a system of equations related to finding the flow values, with B being a basis submatrix of $\tilde{A}$ and $\tilde{x}$ being the vector of flows. The reference to "Ex. 14.12" suggests a worked example illustrating this theorem in the textbook.

Figures

(a)

Slide 17

Content

Spanning Trees and Bases
THEOREM 14.1. A square submatrix of $\tilde{A}$ is a basis if and only if the arcs to which its columns correspond form a spanning tree. *basic variables*

$\tilde{A} = [B \quad N], \quad X = \begin{pmatrix} X_B \\ X_N \end{pmatrix}$

$[B \quad N] \begin{pmatrix} X_B \\ X_N \end{pmatrix} = -b$

$BX_B + NX_N = -b$

$X_B + (B^{-1}N)X_N = -B^{-1}b$

$\Longrightarrow \vec{0} \quad \Longrightarrow \vec{0}$

T - spanning tree

$x_{ij} \neq 0 \quad (ij) \in T$

$x_{ij} = 0 \quad (ij) \notin T$

Description

Slide 17 of 22 in a Linear Programming course (Chapter 14, Network Flows) elaborates on Theorem 14.1, which links spanning trees in a network to bases in the simplex method. The slide shows how the incidence matrix $\tilde{A}$ (modified by removing a row corresponding to a root node, as explained in the previous slide) is partitioned into a basis matrix B and a non-basis matrix N. The flow vector X is partitioned accordingly into basic variables X_B and non-basic variables X_N. The equation $[B \quad N] \begin{pmatrix} X_B \\ X_N \end{pmatrix} = -b$ represents the flow balance constraints. This is then rewritten as $BX_B + NX_N = -b$, and finally solved for the basic variables as $X_B + (B^{-1}N)X_N = -B^{-1}b$. The arrows pointing to zero indicate that in a basic feasible solution associated with a spanning tree T, the non-basic variables (X_N) are zero, and the basic variables (X_B) are determined by solving the system of equations. The final part defines the relationship between the spanning tree T and the flow values: flows on arcs in the tree (x_{ij}) are non-zero, while flows on arcs not in the tree are zero. This highlights the direct correspondence between a spanning tree and a basic feasible solution in network flow problems.

Figures

(a)

Slide 18

Content

Spanning Trees and Bases
Dual Flow
$y_j - y_i + z_{ij} = c_{ij} \quad (i,j) \in A$
$(z_{ij} \geq 0)$
Dual Flow for Spanning Tree (T)
(Complementary Slackness $x_{ij} z_{ij} = 0$)
$(i,j) \in T, \quad x_{ij} \neq 0 \implies z_{ij} = 0$
$y_j - y_i = c_{ij}$
$(i,j) \in A \setminus T, \quad z_{ij} = c_{ij} - (y_j - y_i)$

Description

Slide 18 of 22 in a Linear Programming course (Chapter 14, Network Flows) discusses dual flows in the context of spanning trees. The slide introduces the concept of dual flow with the equation $y_j - y_i + z_{ij} = c_{ij}$, where y_i and y_j are dual variables associated with nodes i and j, z_{ij} is a slack variable, and c_{ij} represents the cost associated with arc (i,j). The constraint $z_{ij} \geq 0$ is also given. The slide then focuses on the dual flow for a spanning tree T, incorporating the complementary slackness condition $x_{ij} z_{ij} = 0$, where x_{ij} represents the flow on arc (i,j). For arcs within the spanning tree ($(i,j) \in T$), a non-zero flow ($x_{ij} \neq 0$) implies zero slack ($z_{ij} = 0$), leading to the equation $y_j - y_i = c_{ij}$. For arcs not in the spanning tree ($(i,j) \in A \setminus T$), the slack variable z_{ij} is expressed as $z_{ij} = c_{ij} - (y_j - y_i)$. This slide connects the primal (flow) and dual variables within the network flow problem, specifically highlighting the relationships when a spanning tree is considered. The underlining emphasizes key concepts and equations.

Figures

(a)

Slide 19

Content

Spanning Trees and Bases
$y_j - y_i = c_{ij}$
For example, let node “g” be the root node in the spanning tree in Figure 14.6.
Starting with it, we compute the dual variables as follows:
root node $\implies y_g = 0$,
across arc (g,e): $y_e - y_g = 19 \implies y_e = 19$,
across arc (g,b): $y_b - y_g = 33 \implies y_b = 33$,
across arc (b,c): $y_c - y_b = 65 \implies y_c = 98$,
across arc (f,b): $y_b - y_f = 48 \implies y_f = -15$,
across arc (f,a): $y_a - y_f = 56 \implies y_a = 41$,
across arc (a,d): $y_d - y_a = 28 \implies y_d = 69$.
Now that we know the dual variables, the dual slacks for the arcs not in the spanning tree T can be computed using
$z_{ij} = y_i + c_{ij} - y_j, \quad (i,j) \notin T$
(which is just the dual feasibility condition solved for z_{ij}). These values are shown on

Description

Slide 19 of 22, in a Linear Programming course (Chapter 14, Network Flows), demonstrates the computation of dual variables in a network flow problem. It builds upon the previous slides' discussion of spanning trees and bases. The slide uses a specific example, selecting node "g" as the root node in a spanning tree (referenced as Figure 14.6, not shown here). The computation of dual variables (y_i) is shown step-by-step, traversing the arcs of the spanning tree. The equation $y_j - y_i = c_{ij}$ is used, where c_{ij} represents the cost of arc (i,j). The root node's dual variable is set to 0 ($y_g = 0$), and the dual variables for other nodes are calculated iteratively based on the costs of the arcs connecting them to the already-computed nodes. Finally, the slide explains how to compute the dual slacks (z_{ij}) for arcs not in the spanning tree using the equation $z_{ij} = y_i + c_{ij} - y_j$, which is described as the dual feasibility condition solved for z_{ij}. The slide's purpose is to illustrate the practical application of the theoretical concepts presented in previous slides, showing how to find dual variables and slacks in a network flow problem given a spanning tree.

Figures

(a)

Slide 20

Content

Spanning Trees and Bases

[scale=0.8] [circle,draw,label=above:a] (a) at (0,3) ; [circle,draw,label=right:b] (b) at (3,1.5) ; [circle,draw,label=left:c] (c) at (1.5,1.5) ; [circle,draw,label=right:d] (d) at (2,2.5) ; [circle,draw,label=right:e] (e) at (4,3) ; [circle,draw,label=left:f] (f) at (0,0) ; [circle,draw,label=right:g] (g) at (4,0) ;

[->, very thick, red] (a) – node[above]32 (e); [->, very thick, red] (a) – node[left]41 (f); [->, very thick, red] (b) – node[right]33 (g); [->, very thick, red] (b) – node[above]6 (d); [->, very thick, red] (c) – node[above]6 (a); [->, very thick, red] (c) – node[below]-5 (f); [->, very thick, red] (d) – node[left]-1 (c); [->, very thick, red] (d) – node[right]21 (e); [->, very thick, red] (f) – node[below]9 (g);

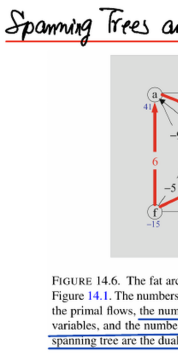
at (0.75,2.25) -9; at (2.25,1.75) 3; at (1.5,0.75) -5; at (3.5,0.75) 3; at (3.75,2.25) 2; at (0.75,0.75) 6; at (2.25,0.75) 9; at (1.5,1.5) 98; at (2,2.5) 69; at (0,3) 41; at (3,1.5) 33; at (1.5,1.5) 98; at (2,2.5) 69; at (4,3) 19; at (0,0) -15; at (4,0) 0;

FIGURE 14.6. The fat arcs show a spanning tree for the network in Figure 14.1. The numbers shown on the arcs of the spanning tree are the primal flows, the numbers shown next to the nodes are the dual variables, and the numbers shown on the arcs not belonging to the spanning tree are the dual slacks.

Description

Slide 20 of 22 in a Linear Programming course (Chapter 14, Network Flows) presents Figure 14.6, illustrating a network flow problem. The figure shows a directed graph with nodes labeled a through g. Thick red arcs represent a spanning tree within the network. Numbers on the arcs represent primal flows (for arcs in the spanning tree) and dual slacks (for arcs not in the spanning tree). Numbers adjacent to each node represent the dual variables associated with that node. The caption explains that the figure demonstrates the relationship between a spanning tree, primal flows, dual variables, and dual slacks in the context of the network flow problem introduced in Figure 14.1 (not shown). The slide visually reinforces the concepts explained in previous slides regarding spanning trees, bases, primal and dual variables, and complementary slackness in network flow optimization.

Figures



(a) A network flow diagram showing a spanning tree. The thick red arcs represent the spanning tree. Numbers on the arcs represent primal flows (for arcs in the spanning tree) and dual slacks (for arcs not in the spanning tree). Numbers next to nodes represent dual variables. The figure is labeled as Figure 14.6.

Slide 21

Content

Spanning Trees and Bases
From duality theory, we know that the current tree solution is optimal if all the flows **are nonnegative** and if all the dual slacks are nonnegative. The tree solution shown in Figure 14.6 satisfies the first condition but not the second. That is, it is primal feasible but not dual feasible. Hence, we can apply the primal simplex method to move from this solution to an optimal one. We take up this task in the next section. [scale=0.8] [circle,draw,label=above:a] (a) at (0,3) 41; [circle,draw,label=right:b] (b) at (3,1.5) 33; [circle,draw,label=left:c] (c) at (1.5,1.5) 98; [circle,draw,label=right:d] (d) at (2,2.5) 69; [circle,draw,label=right:e] (e) at (4,3) 19; [circle,draw,label=left:f] (f) at (0,0) -15; [circle,draw,label=right:g] (g) at (4,0) 0;

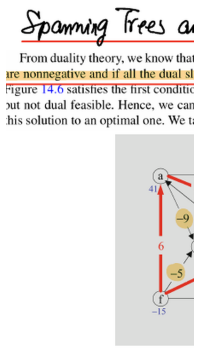
[->, very thick, red] (a) – node[above]32 (e); [->, very thick, red] (a) – node[left]41 (f); [->, very thick, red] (b) – node[right]33 (g); [->, very thick, red] (b) – node[above]6 (d); [->, very thick, red] (c) – node[above]6 (a); [->, very thick, red] (c) – node[below]-5 (f); [->, very thick, red] (d) – node[left]-1 (c); [->, very thick, red] (d) – node[right]21 (e); [->, very thick, red] (f) – node[below]9 (g);

at (0.75,2.25) -9; at (2.25,1.75) 3; at (1.5,0.75) -5; at (3.5,0.75) 3; at (3.75,2.25) 2; at (0.75,0.75) 6; at (2.25,0.75) 9; at (1.5,1.5) 98; at (2,2.5) 69; at (0,3) 41; at (3,1.5) 33; at (1.5,1.5) 98; at (2,2.5) 69; at (4,3) 19; at (0,0) -15; at (4,0) 0;

Description

Slide 21 of 22 in a Linear Programming course (Chapter 14, Network Flows) discusses optimality conditions for network flow problems. It states that a tree solution is optimal if all flows are nonnegative and all dual slacks are nonnegative. Figure 14.6 (shown on the slide) represents a network flow solution that is primal feasible (all flows are nonnegative) but not dual feasible (some dual slacks are negative). The slide concludes that the primal simplex method can be used to improve this solution to reach an optimal one. The figure is a network flow graph with nodes labeled a-g. Thick red arcs represent a spanning tree. Numbers on the arcs represent primal flows (for arcs in the spanning tree) and dual slacks (for arcs not in the spanning tree). Numbers next to nodes represent dual variables. The negative dual slacks indicate that the current solution is not optimal. The underlining highlights the key condition for optimality: all flows must be nonnegative.

Figures



(a) A network flow diagram showing a spanning tree. The thick red arcs represent the spanning tree. Numbers on the arcs represent primal flows (for arcs in the spanning tree) and dual slacks (for arcs not in the spanning tree). Numbers next to nodes represent dual variables. The figure is labeled as Figure 14.6.

Slide 22

Content

Description

Slide 22 of 22 is blank. There is no content on this slide. In the context of a Linear Programming course, this likely indicates the end of the lecture or presentation.

Figures

(a)